

Modelo relacional y consultas de primer orden

Clase 01

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Relaciones

Modelo relacional

Consultas de primer orden

Outline

Relaciones

Modelo relacional

Consultas de primer orden

Tuplas

Definición

- Una ***k*-tupla** $\bar{t}$ es una secuencia ordenada de k elementos:

$$\bar{t} := (a_1, a_2, \dots, a_k)$$

- Cuando k es conocido del contexto, diremos que $\bar{t}$ es una **tupla**.
- Cuando $k = 0$, entonces $\bar{t} = ()$ es la **tupla vacía**.

Ejemplos:

- $\{a, b, c\} = \{c, b, a\}$?
- $(a, b, c) = (c, b, a)$?
- $\{a, a, b\} = \{a, b\}$?
- $(a, a, b) = (a, b)$?
- $() = \emptyset$?
- $\{()\} = \emptyset$?



Tuplas, arreglos, secuencias, vectores y strings

¿cuál es la diferencia?

Tuplas	Arreglos	Secuencias	Vectores	Palabras
$(a_1, \dots, a_n)$	$[a_1, \dots, a_n]$	$a_1, \dots, a_n$	$(a_1, \dots, a_n)$	$a_1 \dots a_n$

Todos estos objetos son lo mismo

(solo que para cada uno
usamos operaciones distintas)

Definiciones útiles

- Para $\bar{t} = (a_1, \dots, a_n)$ y $i \in [n]$ usaremos $\bar{t}[i] = a_i$ para denotar al **i -ésimo elemento** de la tupla $\bar{t}$.
- Para $\bar{t} = (a_1, \dots, a_n)$, usaremos $\{\bar{t}\} = \{a_1, \dots, a_n\}$ para denotar al **conjunto de los elementos** de $\bar{t}$.

Producto cartesiano

Definición

Para los conjuntos $A_1, A_2, \dots, A_k$ definimos el **producto cartesiano**:

$$A_1 \times A_2 \times \dots \times A_k := \{(a_1, a_2, \dots, a_k) \mid a_1 \in A_1, a_2 \in A_2, \dots, a_k \in A_k\}$$

Ejemplo

$$A_1 = \{1, 2\} \quad A_2 = \{\text{Ana}, \text{Ben}\} \quad A_3 = \{101, 102\}$$

$$A_1 \times A_2 \times A_3 = \{(1, \text{Ana}, 101), (1, \text{Ana}, 102), (1, \text{Ben}, 101), (1, \text{Ben}, 102), \\ (2, \text{Ana}, 101), (2, \text{Ana}, 102), (2, \text{Ben}, 101), (2, \text{Ben}, 102)\}$$

Conjunto de **todas las k -tuplas** con elementos en $A_1, \dots, A_k$

Producto cartesiano

Definición

Para los conjuntos $A_1, A_2, \dots, A_k$ definimos el **producto cartesiano**:

$$A_1 \times A_2 \times \dots \times A_k := \{(a_1, a_2, \dots, a_k) \mid a_1 \in A_1, a_2 \in A_2, \dots, a_k \in A_k\}$$

Notación

Para un conjunto A y $k \in \mathbb{N}$ usaremos:

$$\begin{aligned} A^k &= \overbrace{A \times \dots \times A}^{k\text{-veces}} \\ A^0 &= \underbrace{\{ () \}}_{\text{tupla vacía}} \end{aligned}$$

$$\{ () \} \neq \emptyset$$

Relaciones

Definición

Una ***k*-relación** R es un conjunto finito de k -tuplas.

$$R \subseteq A_1 \times \dots \times A_k$$

Ejemplo: 3-relación

Person = $\{(1, \text{Ana}, 101), (1, \text{Ben}, 102),$
 $(2, \text{Ana}, 101), (2, \text{Ben}, 101),$
 $(2, \text{Ben}, 102)\}$

Person		
1	Ana	101
1	Ben	102
2	Ana	101
2	Ben	101
2	Ben	102

¿cuál es la diferencia entre una **relación** y una **tabla**?

Relaciones versus tablas

Definición

Una ***k*-relación** R es un conjunto finito de k -tuplas.

$$R \subseteq A_1 \times \dots \times A_k$$

Diferencias

1. Tablas tienen sus filas **ordenadas** (relaciones no).

Ejemplo

$$\text{Person} = \{(1, \text{Ana}, 102), (2, \text{Ben}, 101)\}$$

Person ₁				≠	Person ₂			
1	Ana	102			2	Ben	101	
2	Ben	101			1	Ana	102	

Relaciones versus tablas

Definición

Una ***k*-relación** R es un conjunto finito de k -tuplas.

$$R \subseteq A_1 \times \dots \times A_k$$

Diferencias

1. Tablas tienen sus filas **ordenadas** (relaciones no).
2. Tablas pueden repetir **filas** (relaciones no).

Ejemplo

Person ₁			$\neq$	Person = { (1, Ana, 102) }
1	Ana	102		
1	Ana	102		

Relaciones versus tablas

Definición

Una **k-relación** R es un conjunto finito de k -tuplas.

$$R \subseteq A_1 \times \dots \times A_k$$

Diferencias

1. Tablas tienen sus filas **ordenadas** (relaciones no).
2. Tablas pueden repetir **filas** (relaciones no).

Durante el curso (y en bases de datos en general)
hablaremos de **tablas y relaciones por igual**

(aunque **conceptualmente**
NO son lo mismo)

Convención / acuerdo

Durante todo el curso:

- Para **elementos** usaremos los símbolos: $a, b, c, d, e, \dots$
- Para **tuplas/arreglos/secuencias** usaremos: $\bar{t}, \bar{s}, \bar{x}, \bar{y}, \bar{z}, \dots$
- Para **conjuntos** usaremos: $A, B, C, \dots$
- Para **relaciones** usaremos: $R, S, T, \dots$
- Para **números/índices** usaremos los símbolos: $i, k, j, l, m, n, \dots$

¡No olvidar!

Outline

Relaciones

Modelo relacional

Consultas de primer orden

Motivación

Queremos almacenar nuestros datos en relaciones como:

Person	Job	City	}	forma
1 Ana 101	1 Guía	101 Lima Peru		
2 Ben 102	2 Chef	102 Stgo Chile	}	contenido
3 Zoe 102	2 Guía			
	2 Poeta			

y hacer consultas como:

1. Encuentra todas las personas que tengan dos o más profesiones.
2. Encuentra todas las ciudades que no tengan personas.

Motivación

Queremos almacenar nuestros datos en relaciones como:

Person	Job	City	
1 Ana 101	1 Guía	101 Lima Peru	} forma = esquema
2 Ben 102	2 Chef	102 Stgo Chile	
3 Zoe 102	2 Guía		} contenido = datos
	2 Poeta		

y hacer consultas como:

1. Encuentra todas las ^{Person} personas que tengan dos o más ^{Job} profesiones.
2. Encuentra todas las ^{City} ciudades que no tengan ^{Person} personas.

Queremos que el **lenguaje** se refiera a la **forma** y no al **contenido**

Modelo relacional

Sea **Data** un conjunto de **datos fijo**.

Ejemplo de conjunto de datos

$$\begin{aligned} \mathbf{Data} = \{ & \underbrace{0, 1, 2, 3, \dots}_{\text{Ints}} \\ & \underbrace{a, b, c, \dots, z, A, B, C, \dots Z,}_{\text{Char}} \\ & \underbrace{\text{Ana, Ben, Zoe, } \dots}_{\text{Strings}} \\ & \underbrace{01/01/1945, 10/01/1945, \dots}_{\text{Dates}} \} \end{aligned}$$

En general, **Data** son todos los posibles datos soportados en el sistema.

Modelo relacional: esquema

Sea **Data** un conjunto de **datos fijo**.

Definición

Un **esquema** (de bases de datos) es un par $\mathbb{S} = (\mathbb{N}, \text{ar})$ tal que:

- $\mathbb{N}$ es un conjunto de **nombres** de relaciones y
- $\text{ar} : \mathbb{N} \rightarrow \mathbb{N}$ es una función que asigna a cada nombre una **aridad**.

Ejemplo de esquema

Person	Job	City	} $\mathbb{S} = (\mathbb{N}, \text{ar})$
1 Ana 101	1 Guía	101 Lima Peru	
2 Ben 102	2 Chef	102 Stgo Chile	
3 Zoe 102	2 Guía		
	2 Poeta		

$$\begin{aligned}\mathbb{N} = \{ \text{Person}, \text{Job}, \text{City} \} \quad \text{ar}(\text{Person}) &= 3 \\ &\text{ar}(\text{Job}) = 2 \\ &\text{ar}(\text{City}) = 3\end{aligned}$$

Modelo relacional: base de datos

Sea **Data** un conjunto de **datos** fijo.

Definición

Un **esquema** (de bases de datos) es un par $\mathbb{S} = (\mathbb{N}, \text{ar})$ tal que:

- $\mathbb{N}$ es un conjunto de **nombres** de relaciones y
- $\text{ar} : \mathbb{N} \rightarrow \mathbb{N}$ es una función que asigna a cada nombre una **aridad**.

Sea **Rel** el conjunto de **todas las relaciones** sobre **Data** :

$$\mathbf{Rel} = \{R \mid R \subseteq \mathbf{Data}^k \text{ y } |R| < \infty \text{ para algún } k \in \mathbb{N}\}$$

Definición

Sea $\mathbb{S} = (\mathbb{N}, \text{ar})$ un esquema. Una **base de datos** D de $\mathbb{S}$ es una función:

$$D : \mathbb{N} \rightarrow \mathbf{Rel}$$

tal que $D(R) \subseteq \mathbf{Data}^k$ con $k = \text{ar}(R)$.

Modelo relacional: base de datos

Definición

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema. Una **base de datos** D de $\mathbb{S}$ es una función:

$$D : \mathbf{N} \rightarrow \mathbf{Rel}$$

tal que $D(R) \subseteq \mathbf{Data}^k$ con $k = \text{ar}(R)$.

Ejemplo de base de datos

$$\begin{aligned} \mathbf{N} = \{ \text{Person}, \text{Job}, \text{City} \} \quad & \text{ar}(\text{Person}) = 3 \\ & \text{ar}(\text{Job}) = 2 \\ & \text{ar}(\text{City}) = 3 \end{aligned}$$

$$\begin{aligned} D(\text{Person}) &= \{(1, \text{Ana}, 101), (2, \text{Ben}, 102), (3, \text{Zoe}, 102)\} \\ D(\text{Job}) &= \{(1, \text{Guía}), (2, \text{Chef}), (2, \text{Guía}), (2, \text{Poeta})\} \\ D(\text{City}) &= \{(101, \text{Lima}, \text{Peru}), (102, \text{Stgo}, \text{Chile})\} \end{aligned}$$

Modelo relacional: base de datos

Ejemplo de base de datos (continuación)

$$\begin{aligned} N = \{\text{Person}, \text{Job}, \text{City}\} \quad \text{ar}(\text{Person}) &= 3 \\ \text{ar}(\text{Job}) &= 2 \\ \text{ar}(\text{City}) &= 3 \end{aligned}$$

$$D(\text{Person}) = \{(1, \text{Ana}, 101), (2, \text{Ben}, 102), (3, \text{Zoe}, 102)\}$$

$$D(\text{Job}) = \{(1, \text{Guía}), (2, \text{Chef}), (2, \text{Guía}), (2, \text{Poeta})\}$$

$$D(\text{City}) = \{(101, \text{Lima}, \text{Peru}), (102, \text{Stgo}, \text{Chile})\}$$

Person	Job	City	} $\mathbb{S} = (N, \text{ar})$
1 Ana 101	1 Guía	101 Lima Peru	
2 Ben 102	2 Chef	102 Stgo Chile	} D
3 Zoe 102	2 Guía		
	2 Poeta		

Modelo relacional: dominio

Definición

Para una bases de datos D de $\mathbb{S} = (\mathbb{N}, \text{ar})$, definimos el **dominio** de D como:

$$\text{Dom}(D) := \bigcup_{R \in \mathbb{N}} \bigcup_{\bar{t} \in D(R)} \{\bar{t}\}$$

En otras palabras, **todos los datos** que aparecen en relaciones de D .

Ejemplo de dominio

$$D(\text{Person}) = \{(1, \text{Ana}, 101), (2, \text{Ben}, 102), (3, \text{Zoe}, 102)\}$$

$$D(\text{Job}) = \{(1, \text{Guía}), (2, \text{Chef}), (2, \text{Guía}), (2, \text{Poeta})\}$$

$$D(\text{City}) = \{(101, \text{Lima}, \text{Peru}), (102, \text{Stgo}, \text{Chile})\}$$

$$\begin{aligned} \text{Dom}(D) = \{ & 1, 2, 3, \text{Ana}, \text{Ben}, \text{Zoe}, 101, 102, \\ & \text{Guía}, \text{Chef}, \text{Poeta}, \\ & \text{Lima}, \text{Peru}, \text{Stgo}, \text{Chile} \} \end{aligned}$$

El dominio de D será **útil** para definir la semántica de las consultas.

Outline

Relaciones

Modelo relacional

Consultas de primer orden

Motivación

Queremos almacenar nuestros datos en relaciones como:

Person			Job		City		
1	Ana	101	1	Guía	101	Lima	Peru
2	Ben	102	2	Chef	102	Stgo	Chile
3	Zoe	102	2	Guía			
			2	Poeta			

y hacer consultas como:

1. Encuentra todas las personas que tengan dos o más profesiones.
2. Encuentra todas las ciudades que no tengan personas.

Para declarar las consultas veremos distintos **lenguajes de consultas**

... partiendo hoy por **lógica de primer orden**.

Datos, variables y constantes

Definición

- Sea **Data** un conjunto de **datos fijo**.
- Sea **Var** el conjunto de todas las **variables** con $\mathbf{Data} \cap \mathbf{Var} = \emptyset$.

Notación

- Elementos en **Data** también los llamaremos **constantes**.
- Elementos en $\mathbf{Data} \cup \mathbf{Var}$ los llamaremos **términos**.
- Usaremos $a, b, c, \dots$ para denotar **constantes** en **Data**.
- Usaremos $x, y, z, \dots$ para denotar **variables** en **Var**.
- Usaremos $t, r, s, \dots$ para denotar **términos** en $\mathbf{Data} \cup \mathbf{Var}$.

Sintaxis de lógica de primer orden

Sea **Data** y **Var** los conj. de **constantes** y **variables** tal que $\mathbf{Data} \cap \mathbf{Var} = \emptyset$.

Definición (sintaxis)

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema.

Una **formula** φ sobre $\mathbb{S}$ de **lógica de primer orden** se define recursivamente:

φ	$:=$	$x = a$	(igualdad con constante)	}	Casos base
		$x = y$	(igualdad entre variables)		
		$R(t_1, \dots, t_k)$	(relaciones)		
		$\varphi_1 \vee \varphi_2$	(disyunción)	}	Casos inductivos
		$\varphi_1 \wedge \varphi_2$	(conjunción)		
		$\neg \psi$	(negación)		
		$\exists x. \psi$	(cuantificación existencial)		
		$\forall x. \psi$	(cuantificación universal)		

con $x, y \in \mathbf{Var}$, $a \in \mathbf{Data}$, $R \in \mathbf{N}$ con $k = \text{ar}(R)$, y $t_1, \dots, t_k \in \mathbf{Data} \cup \mathbf{Var}$,

con φ_1, φ_2 y ψ formulas sobre $\mathbb{S}$.

Sintaxis de lógica de primer orden

Sea **Data** y **Var** los conj. de **constantes** y **variables** tal que $\mathbf{Data} \cap \mathbf{Var} = \emptyset$.

Definición (sintaxis)

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema.

Una **formula** φ sobre $\mathbb{S}$ de **lógica de primer orden** se define recursivamente:

φ	$:=$	$x = a$	(igualdad con constante)	}	Formulas atómicas
		$x = y$	(igualdad entre variables)		
		$R(t_1, \dots, t_k)$	(relaciones)		
		$\varphi_1 \vee \varphi_2$	(disyunción)	}	Formulas compuestas
		$\varphi_1 \wedge \varphi_2$	(conjunción)		
		$\neg \psi$	(negación)		
		$\exists x. \psi$	(cuantificación existencial)		
		$\forall x. \psi$	(cuantificación universal)		

con $x, y \in \mathbf{Var}$, $a \in \mathbf{Data}$, $R \in \mathbf{N}$ con $k = \text{ar}(R)$, y $t_1, \dots, t_k \in \mathbf{Data} \cup \mathbf{Var}$,

con φ_1, φ_2 y ψ formulas sobre $\mathbb{S}$.

Sintaxis de lógica de primer orden

φ	$:=$	$x = a$	(igualdad con constante)
		$x = y$	(igualdad entre variables)
		$R(t_1, \dots t_k)$	(relaciones)
		$\varphi_1 \vee \varphi_2$	(disyunción)
		$\varphi_1 \wedge \varphi_2$	(conjunción)
		$\neg \psi$	(negación)
		$\exists x. \psi$	(cuantificación existencial)
		$\forall x. \psi$	(cuantificación universal)

Ejemplos de formulas

$$\mathbb{S} = (\mathbb{N}, \text{ar}) \quad \mathbb{N} = \left\{ \overbrace{\text{Person}}^{\text{ar}=3}, \overbrace{\text{Job}}^{\text{ar}=2}, \overbrace{\text{City}}^{\text{ar}=3} \right\}$$

1. $\text{Person}(x, \text{Ben}, y)$
2. $\text{Person}(x, \text{Ben}, y) \wedge y = 102$
3. $\exists x. \exists y. \text{Person}(x, \text{Ben}, y) \wedge y = 102 \wedge \text{Job}(x, z)$

Suponemos la precedencia, asociatividad y conmutatividad usual de $\neg$, $\wedge$, $\vee$.

Variables libres y constantes de una formula

Definición

Se define los conjuntos $FV(\varphi)$ y $Const(\varphi)$

de las **variables libres** (free-variables) y **constantes** de φ recursivamente:

$$FV(x = a) := \{x\}$$

$$FV(x = y) := \{x, y\}$$

$$FV(R(\bar{t})) := \{\bar{t}\} \cap \mathbf{Var}$$

$$FV(\varphi \vee \psi) := FV(\varphi) \cup FV(\psi)$$

$$FV(\varphi \wedge \psi) := FV(\varphi) \cup FV(\psi)$$

$$FV(\neg\varphi) := FV(\varphi)$$

$$FV(\exists x. \varphi) := FV(\varphi) \setminus \{x\}$$

$$FV(\forall x. \varphi) := FV(\varphi) \setminus \{x\}$$

$$Const(x = a) := \{a\}$$

$$Const(x = y) := \emptyset$$

$$Const(R(\bar{t})) := \{\bar{t}\} \cap \mathbf{Data}$$

$$Const(\varphi \vee \psi) := Const(\varphi) \cup Const(\psi)$$

$$Const(\varphi \wedge \psi) := Const(\varphi) \cup Const(\psi)$$

$$Const(\neg\varphi) := Const(\varphi)$$

$$Const(\exists x. \varphi) := Const(\varphi)$$

$$Const(\forall x. \varphi) := Const(\varphi)$$

Variables libres y constantes de una formula

$$\text{FV}(x = a) := \{x\}$$

$$\text{FV}(x = y) := \{x, y\}$$

$$\text{FV}(R(\bar{t})) := \{\bar{t}\} \cap \mathbf{Var}$$

$$\text{FV}(\varphi \vee \psi) := \text{FV}(\varphi) \cup \text{FV}(\psi)$$

$$\text{FV}(\varphi \wedge \psi) := \text{FV}(\varphi) \cup \text{FV}(\psi)$$

$$\text{FV}(\neg \varphi) := \text{FV}(\varphi)$$

$$\text{FV}(\exists x. \varphi) := \text{FV}(\varphi) \setminus \{x\}$$

$$\text{FV}(\forall x. \varphi) := \text{FV}(\varphi) \setminus \{x\}$$

$$\text{Const}(x = a) := \{a\}$$

$$\text{Const}(x = y) := \emptyset$$

$$\text{Const}(R(\bar{t})) := \{\bar{t}\} \cap \mathbf{Data}$$

$$\text{Const}(\varphi \vee \psi) := \text{Const}(\varphi) \cup \text{Const}(\psi)$$

$$\text{Const}(\varphi \wedge \psi) := \text{Const}(\varphi) \cup \text{Const}(\psi)$$

$$\text{Const}(\neg \varphi) := \text{Const}(\varphi)$$

$$\text{Const}(\exists x. \varphi) := \text{Const}(\varphi)$$

$$\text{Const}(\forall x. \varphi) := \text{Const}(\varphi)$$

Ejemplos de variables libres y constantes

1. $\varphi_1 := \text{Person}(x, \text{Ben}, y)$

$$\text{FV}(\varphi_1) = \{x, y\} \quad \text{Const}(\varphi_1) = \{\text{Ben}\}$$

2. $\varphi_2 := \exists x. \exists y. \text{Person}(x, \text{Ben}, y) \wedge y = 102 \wedge \text{Job}(x, z)$

$$\text{FV}(\varphi_2) = \{z\} \quad \text{Const}(\varphi_2) = \{\text{Ben}, 102\}$$

Variables libres y constantes de una formula

$$\text{FV}(x = a) := \{x\}$$

$$\text{FV}(x = y) := \{x, y\}$$

$$\text{FV}(R(\bar{t})) := \{\bar{t}\} \cap \mathbf{Var}$$

$$\text{FV}(\varphi \vee \psi) := \text{FV}(\varphi) \cup \text{FV}(\psi)$$

$$\text{FV}(\varphi \wedge \psi) := \text{FV}(\varphi) \cup \text{FV}(\psi)$$

$$\text{FV}(\neg \varphi) := \text{FV}(\varphi)$$

$$\text{FV}(\exists x. \varphi) := \text{FV}(\varphi) \setminus \{x\}$$

$$\text{FV}(\forall x. \varphi) := \text{FV}(\varphi) \setminus \{x\}$$

$$\text{Const}(x = a) := \{a\}$$

$$\text{Const}(x = y) := \emptyset$$

$$\text{Const}(R(\bar{t})) := \{\bar{t}\} \cap \mathbf{Data}$$

$$\text{Const}(\varphi \vee \psi) := \text{Const}(\varphi) \cup \text{Const}(\psi)$$

$$\text{Const}(\varphi \wedge \psi) := \text{Const}(\varphi) \cup \text{Const}(\psi)$$

$$\text{Const}(\neg \varphi) := \text{Const}(\varphi)$$

$$\text{Const}(\exists x. \varphi) := \text{Const}(\varphi)$$

$$\text{Const}(\forall x. \varphi) := \text{Const}(\varphi)$$

Ejemplos de variables libres y constantes (continuación)

$$2. \varphi_2 := \exists x. \exists y. \text{Person}(x, \text{Ben}, y) \wedge y = 102 \wedge \text{Job}(x, z)$$

$$\text{FV}(\varphi_2) = \{z\}$$

$$\text{Const}(\varphi_2) = \{\text{Ben}, 102\}$$

$$3. \varphi_3 := \forall x. \exists y. \text{Person}(x, \text{Ana}, y) \wedge \text{Job}(x, \text{Guía})$$

$$\text{FV}(\varphi_3) = \emptyset$$

$$\text{Const}(\varphi_3) = \{\text{Ana}, \text{Guía}\}$$

Semántica de lógica de primer orden

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema, φ una formula y D una base de datos de $\mathbb{S}$.

Una **asignación** α de φ sobre D es una función:

$$\alpha : \text{FV}(\varphi) \rightarrow \text{Dom}(D) \cup \text{Const}(\varphi)$$

que a cada variable libre de φ le asigna un dato de D o constante de φ .

Semántica de lógica de primer orden

Definimos que D y α **satisfacen** φ , $(D, \alpha) \models \varphi$, inductivamente como:

$$\begin{array}{lll} (D, \alpha) \models (x = a) & \text{ssi} & \alpha(x) = a \\ (D, \alpha) \models (x = y) & \text{ssi} & \alpha(x) = \alpha(y) \\ (D, \alpha) \models R(t_1, \dots, t_k) & \text{ssi} & (\alpha(t_1), \dots, \alpha(t_k)) \in D(R) \\ (D, \alpha) \models \varphi_1 \vee \varphi_2 & \text{ssi} & (D, \alpha) \models \varphi_1 \text{ o } (D, \alpha) \models \varphi_2 \\ (D, \alpha) \models \varphi_1 \wedge \varphi_2 & \text{ssi} & (D, \alpha) \models \varphi_1 \text{ y } (D, \alpha) \models \varphi_2 \\ (D, \alpha) \models \neg \varphi & \text{ssi} & \text{no ocurre que } (D, \alpha) \models \varphi \end{array}$$

Por **simplificación**, para todo $a \in \mathbf{Data}$ suponemos que $\alpha(a) = a$.

Semántica de lógica de primer orden

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema, φ una formula y D una base de datos de $\mathbb{S}$.

Una **asignación** α de φ sobre D es una función:

$$\alpha : \text{FV}(\varphi) \rightarrow \text{Dom}(D) \cup \text{Const}(\varphi)$$

que a cada variable libre de φ le asigna un dato de D o constante de φ .

Semántica de lógica de primer orden

Definimos que D y α **satisfacen** φ , $(D, \alpha) \models \varphi$, inductivamente como:

$$(D, \alpha) \models \exists x. \varphi \quad \text{ssi} \quad (D, \alpha_{x/a}) \models \varphi \quad \text{para algún } a \in \text{Dom}(D) \cup \text{Const}(\varphi)$$

$$(D, \alpha) \models \forall x. \varphi \quad \text{ssi} \quad (D, \alpha_{x/a}) \models \varphi \quad \text{para todo } a \in \text{Dom}(D) \cup \text{Const}(\varphi)$$

donde $\alpha_{x/a}$ es la **asignación extendida** de α tal que para todo y :

$$\alpha_{x/a}(y) = \begin{cases} a & \text{si } y = x \\ \alpha(y) & \text{si } y \neq x \end{cases}$$

Semántica de lógica de primer orden

$$\begin{aligned} D, \alpha \models x = a & \text{ ssi } \alpha(x) = a \\ (D, \alpha) \models R(t_1, \dots, t_k) & \text{ ssi } (\alpha(t_1), \dots, \alpha(t_k)) \in D(R) \\ (D, \alpha) \models \varphi_1 \wedge \varphi_2 & \text{ ssi } (D, \alpha) \models \varphi_1 \text{ y } (D, \alpha) \models \varphi_2 \\ (D, \alpha) \models \exists x. \varphi & \text{ ssi } (D, \alpha_{x/a}) \models \varphi \\ & \text{ para algún } a \in \text{Dom}(D) \cup \text{Const}(\varphi) \end{aligned}$$

Ejemplos de semántica

$D :$	Person	Job	City
1	Ana 101	1 Guía	101 Lima Peru
2	Ben 102	2 Chef	102 Stgo Chile
3	Zoe 102	2 Guía	
		2 Poeta	

1. $(D, [x \mapsto 2, y \mapsto 102]) \models \text{Person}(x, \text{Ben}, y) ?$ ✓
2. $(D, [z \mapsto \text{Chef}]) \models \exists x. \exists y. \text{Person}(x, \text{Ben}, y) \wedge y = 102 \wedge \text{Job}(x, z) ?$ ✓

Consultas de primer orden

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema y D una base de datos de $\mathbb{S}$.

Definición (sintaxis)

Una **consulta de primer orden** Q es una expresión de la forma:

$$Q(x_1, \dots, x_n) \leftarrow \varphi$$

tal que φ es una formula y $\text{FV}(\varphi) \subseteq \{x_1, \dots, x_n\}$.

Todas las variables libres de φ deben aparecer en la lista $x_1, \dots, x_n$.

Ejemplos de consultas

1. $Q_1(x, y) \leftarrow \text{Person}(x, \text{Ben}, y)$
2. $Q_2(z) \leftarrow \exists x. \exists y. \text{Person}(x, \text{Ben}, y) \wedge y = 102 \wedge \text{Job}(x, z)$

Consultas de primer orden

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema y D una base de datos de $\mathbb{S}$.

Definición (sintaxis)

Una **consulta de primer orden** Q es una expresión de la forma:

$$Q(x_1, \dots, x_n) \leftarrow \varphi$$

tal que φ es una fórmula y $\text{FV}(\varphi) \subseteq \{x_1, \dots, x_n\}$.

Definición (semántica)

Para una consulta de primer orden $Q(x_1, \dots, x_n) \leftarrow \varphi$, se define la **relación**:

$$Q(D) = \{(\alpha(x_1), \dots, \alpha(x_n)) \mid (D, \alpha) \models \varphi\}$$

Una consulta de primer orden **recibe relaciones y entrega una relación.**

Consultas de primer orden

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema y D una base de datos de $\mathbb{S}$.

Definición (semántica)

Para una consulta de primer orden $Q(x_1, \dots, x_n) \leftarrow \varphi$, se define la relación:

$$Q(D) = \{(\alpha(x_1), \dots, \alpha(x_n)) \mid (D, \alpha) \models \varphi\}$$

Ejemplos de semántica

$D:$	<u>Person</u>	<u>Job</u>	<u>City</u>
1	Ana 101	1 Guía	101 Lima Peru
2	Ben 102	2 Chef	102 Stgo Chile
3	Zoe 102	2 Guía	
		2 Poeta	

1. $Q_1(x, y) \leftarrow \text{Person}(x, \text{Ben}, y)$

$$Q_1(D) = \{(2, 102)\}$$

Consultas de primer orden

Sea $\mathbb{S} = (\mathbf{N}, \text{ar})$ un esquema y D una base de datos de $\mathbb{S}$.

Definición (semántica)

Para una consulta de primer orden $Q(x_1, \dots, x_n) \leftarrow \varphi$, se define la relación:

$$Q(D) = \{(\alpha(x_1), \dots, \alpha(x_n)) \mid (D, \alpha) \models \varphi\}$$

Ejemplos de semántica (continuación)

D :	<u>Person</u>	<u>Job</u>	<u>City</u>
1	Ana 101	1 Guía	101 Lima Peru
2	Ben 102	2 Chef	102 Stgo Chile
3	Zoe 102	2 Guía	
		2 Poeta	

2. $Q_2(z) \leftarrow \exists x. \exists y. \text{Person}(x, \text{Ben}, y) \wedge y = 102 \wedge \text{Job}(x, z)$

$$Q_2(D) = \{(\text{Chef}), (\text{Guía}), (\text{Poeta})\}$$

Consultas de primer orden

Ejercicio: Entregue una consulta de primer orden

<i>D :</i>	Person	Job	City
	1 Ana 101	1 Guía	101 Lima Peru
	2 Ben 102	2 Chef	102 Stgo Chile
	3 Zoe 102	2 Guía	
		2 Poeta	

3. Encuentra todas las personas que tengan dos o más profesiones.
4. Encuentra todas las ciudades que no tengan personas.